Shielded Payments for Decentralized Cloud Services: Anonymous Pay-Per-Use via Cross-Chain Shielded Pools, Prepaid Credits, and Rate-Limiting Nullifiers

Drew Stone
drew@tangle.tools
Tangle Network

March 2026

Abstract

We present a privacy-preserving payment system for decentralized cloud services—LLM inference, image generation, compute—where users pay anonymously and operators serve requests without learning the payer's identity. The system composes three layers: (1) a cross-chain UTXO-based shielded pool [1] with variable-amount JoinSplit transactions and multi-stablecoin wrapping, (2) a stateless gateway contract that bridges anonymous partial withdrawals to Tangle's Blueprint service lifecycle, and (3) a prepaid credit scheme where a single zero-knowledge proof funds an account from which many jobs are authorized via EIP-712 signatures. We describe the deployed system (v1) and a planned upgrade to per-request unlinkability via Rate-Limiting Nullifiers [3] composed with the existing audited VAnchor circuit (v2). The v1 construction requires no modifications to Tangle's audited core contracts and introduces under 1,000 lines of new Solidity.

Contents

1 Introduction **1**
1.1 Contributions . . . 2

2 Preliminaries **2**

3 System Architecture (v1) **3**
3.1 UTXO Model and Partial Withdrawals . . . 3
3.2 Shielded Gateway . . . 3
3.3 Shielded Credits . . . 4

4 Privacy Analysis **4**
4.1 Operator Knowledge Boundary . . . 4
4.2 Unlinkability Across Sessions . . . 4
4.3 Limitations of v1 . . . 4

5 Upgrade Path: Per-Request Unlinkability via RLN **5**
5.1 Motivation . . . 5
5.2 Rate-Limiting Nullifiers . . . 5
5.3 Circuit Composition . . . 5
5.4 v2 Protocol Flow . . . 5
5.5 v1 → v2 Comparison . . . 6

6 Use Cases **6**

7 Cross-Chain Architecture 6

8 Cost Analysis 7

9 Implementation 7

1 Introduction

Every major AI inference provider—OpenAI, Anthropic, Google—requires account creation, identity verification, and traceable payment. A developer querying an LLM for code review, legal analysis, or medical information creates a permanent record linking their identity to every prompt. Decentralized alternatives inherit the same problem: on-chain payments link wallet addresses to service consumption with full public auditability.

This is not hypothetical. Inference providers can profile users by query content, timing, and volume. Employers can monitor which employees use AI assistance. Competitors can observe which organizations are training custom models. The payment trail is the surveillance vector.

Buterin and Crapis [2] proposed *ZK API Usage Credits*: deposit into a smart contract, generate a zero-knowledge proof per API call to authorize payment without revealing identity. The proposal achieves strong privacy—per-request unlinkability via Rate-Limiting Nullifiers—but requires a Groth16 proof per API call (∼640ms generation, ∼300k gas on-chain).

We present a two-phase approach that ships immediately (v1) while providing a clear upgrade path to per-request unlinkability (v2):

v1 (deployed): One ZK proof funds a pseudonymous credit account. Many ECDSA signatures authorize individual job payments (∼50k gas each). Requests within a credit session share a pseudonym; sessions are unlinkable.

v2 (planned): Each request carries its own RLN-based ZK proof, verified off-chain by the operator. Zero on-chain transactions per request. Full per-request unlinkability. The RLN circuit [3] composes with the audited VAnchor circuit [1, 5] with ∼100 new constraints.

1.1 Contributions

1. A UTXO-based privacy model with partial withdrawals producing unlinkable change notes, enabling incremental spending from a single deposit.
2. A two-phase payment scheme: one ZK proof amortized over many ECDSA-signed payments (v1), upgradeable to per-request RLN proofs verified off-chain (v2).
3. A gateway architecture bridging VAnchor shielded pools to Tangle's service lifecycle with zero core protocol modifications.
4. Multi-asset wrapping (USDC, USDT, DAI → single pool token) maximizing the anonymity set.
5. A cross-chain bridge via LayerZero V2 with dynamic edge addition, supporting deposits on 8 chains.
6. An audit-preserving circuit composition strategy where the v2 upgrade reuses audited sub-circuits (VAnchor [5], RLN [4]) with ∼100 lines of new glue constraints.

2 Preliminaries

We adopt notation from the Webb Protocol [1]. Let $H : \mathbb{F}_p^* \to \mathbb{F}_p$ denote Poseidon [9], a zkSNARK-friendly hash over the BN254 scalar field.

Definition 1 (UTXO Commitment)**.** A commitment to a UTXO with chain identifier i, value v, public key $\mathsf{pk} = H(\mathsf{sk})$, and blinding factor $r \xleftarrow{\$} \mathbb{F}_p$ is:

$$\mathsf{cm} = H(i, v, \mathsf{pk}, r)$$

Definition 2 (Nullifier)**.** The nullifier for commitment cm at Merkle path index j, with hash-based signature $\sigma = H(\mathsf{sk}, j, \mathsf{cm})$, is:

$$\mathsf{sn} = H(\mathsf{cm}, j, \sigma)$$

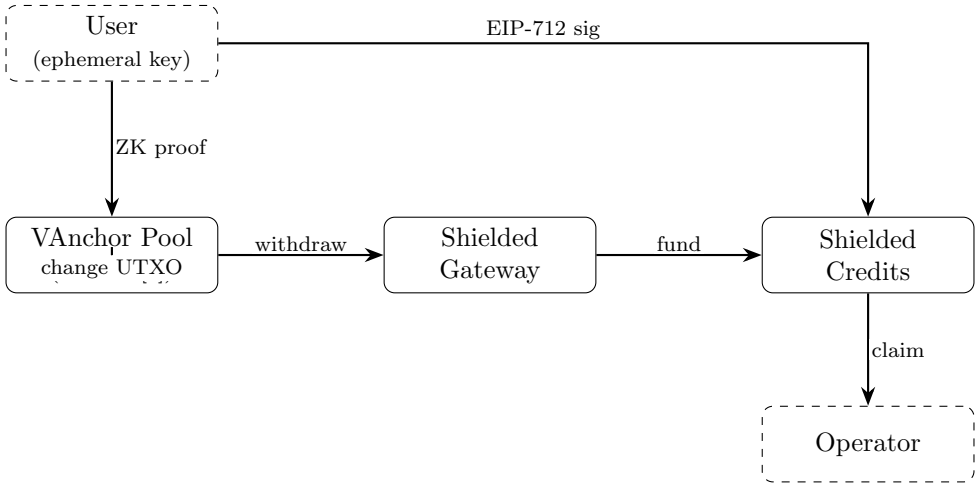
Binding σ to the secret key prevents nullifier grinding by parties who observe commitments but do not hold sk.

Definition 3 (Cross-Chain Merkle Membership)**.** Given Merkle roots $\mathbf{MR} = \{\mathsf{MR}_1, \ldots, \mathsf{MR}_k\}$ from k connected anchor instances, commitment cm satisfies membership if:

$$\mathsf{reconstruct}(\mathsf{cm}, \mathsf{path}(\mathsf{cm}), H) \in \mathbf{MR}$$

The circuit is compiled with a fixed k (currently $k = 8$). Unoccupied edge slots are filled with a deterministic zero hash. New chains are added on-chain via `addEdge` without circuit recompilation or redeployment.

3 System Architecture (v1)



3.1 UTXO Model and Partial Withdrawals

The VAnchor [1] implements a UTXO-based shielded pool with JoinSplit transactions (2-in-2-out or 16-in-2-out). Unlike fixed-denomination mixers (e.g., Tornado Cash [6]), each UTXO carries an arbitrary value, enabling partial withdrawals:

1. User holds $\mathsf{UTXO}(v)$ in the pool (e.g., $v = 100$ USDC).
2. User generates a Groth16 proof spending $\mathsf{UTXO}(100)$ with outputs:
   - Withdrawal of 10 to the gateway ($\mathsf{extAmount} = -10$)
   - Change $\mathsf{UTXO}(90)$ re-shielded in the pool
3. The circuit enforces conservation: $\mathsf{public_amount} + \sum v_{\mathsf{in}} = \sum v_{\mathsf{out}}$.
4. The spent UTXO's nullifier is marked on-chain. The change UTXO has a fresh commitment with fresh blinding—**unlinkable to the original deposit or prior withdrawals**.

The user repeats this process for each credit session. Each withdrawal consumes a different UTXO, produces a different nullifier, and creates a different change commitment. An observer sees n withdrawals from the pool but cannot determine whether they originate from one depositor or n different depositors.

3.2 Shielded Gateway

The `ShieldedGateway` is a stateless proxy between the VAnchor and Tangle's service layer. It executes a VAnchor withdrawal (validating the Groth16 proof and marking nullifiers on-chain), receives tokens, and atomically forwards them to one of:

- `shieldedRequestService`: Create a Tangle service instance with the gateway as owner and ephemeral addresses as permitted callers.
- `shieldedFundService`: Top up an existing service's escrow.
- `shieldedFundCredits`: Fund a prepaid credit account (Section 3.3).

Property 1 (Atomicity)**.** The gateway holds no funds beyond a single transaction. Token receipt and forwarding occur in the same call. Failure at any point reverts the entire transaction, including the VAnchor withdrawal.

Property 2 (Non-Linkability)**.** The gateway stores no persistent state mapping nullifiers to service or credit identifiers. The on-chain trace reveals only the flow `VAnchor` → `Gateway` → `Credits`, not which depositor initiated it.

3.3 Shielded Credits

The core v1 contribution: a prepaid credit system that amortizes one ZK proof across many service invocations.

Definition 4 (Credit Account)**.** Identified by $\mathsf{C} = \mathsf{keccak256}(\mathsf{pk}_s \| \mathsf{salt})$ where pk_s is an ephemeral ECDSA public key and $\mathsf{salt} \xleftarrow{\$} \{0,1\}^{256}$. Account state: $(\mathsf{pk}_s, \mathsf{token}, \mathsf{balance}, \mathsf{nonce})$.

Definition 5 (Spend Authorization)**.** An EIP-712 [8] typed signature over:

$$\mathsf{SpendAuth} = \mathsf{Sign}_{\mathsf{sk}_s}\big(\mathsf{C},\ \mathsf{serviceId},\ \mathsf{jobIndex},\ \mathsf{amount},\ \mathsf{operator},\ \mathsf{nonce},\ \mathsf{expiry}\big)$$

The designated $\mathsf{operator}$ is the only address that can claim payment. Claims after expiry are rejected; unclaimed funds can be reclaimed by anyone via ReclaimExpiredAuth.

The credit lifecycle proceeds as: FundCredits (pull tokens, initialize account) → AuthorizeSpend (verify signature, deduct balance, store pending claim) → ClaimPayment (transfer to operator, require $\mathsf{msg.sender} = \mathsf{operator}$ and $\mathsf{timestamp} \leq \mathsf{expiry}$).

4 Privacy Analysis

4.1 Operator Knowledge Boundary

Information	Operator	Chain observer
User's wallet address	×	×
Deposit amount / total shielded balance	×	×
Which deposit funded this credit account	×	×
Credit commitment C	✓	✓
Payment amount per job	✓	✓
Request content (prompt, image, query)	✓	×
Linkage between jobs within same C	✓	✓
Linkage between different credit accounts	×	×

The operator knows *what* was requested and *how much* was paid, but not *who* is paying. This is the "cash at a store" model: the merchant sees the transaction but cannot trace it to a bank account.

4.2 Unlinkability Across Sessions

When credit account C_1 is depleted, the user generates a new partial withdrawal from their shielded UTXO, creating C_2 with a fresh ephemeral key. The two accounts are cryptographically unlinkable because they use different ephemeral keys, different commitments, different nullifiers, and the connecting change UTXO has fresh blinding. The anonymity set is the total number of depositors in the pool.

4.3 Limitations of v1

Intra-session linkability. Multiple spends from the same C are linkable—the commitment is a pseudonym. Full per-request unlinkability requires a ZK proof per request (addressed in v2, Section 5).

Operator sees request content. Payment privacy does not imply computation privacy. TEE-based execution would address this orthogonally.

Anonymity set dependence. Privacy degrades with few depositors. Multi-asset wrapping and cross-chain deposits are designed to grow the set.

No IP-level privacy. Network-layer anonymity (Tor, Nym) is orthogonal and composable.

5 Upgrade Path: Per-Request Unlinkability via RLN

5.1 Motivation

The v1 system links requests within a credit session. For sensitive workloads—medical queries, legal analysis, investigative journalism—even pseudonymous linkability may be unacceptable. The v2 upgrade eliminates intra-session linkability entirely.

5.2 Rate-Limiting Nullifiers

Rate-Limiting Nullifiers (RLN) [3] use Shamir's Secret Sharing to enforce rate limits without identity:

- Each request in epoch e produces nullifier $\mathsf{rlnNull} = H(\mathsf{sk}, e)$ and reveals one Shamir share: $\mathsf{share} = \mathsf{sk} + a \cdot H(\mathsf{message})$ where $a = H(\mathsf{sk}, e)$.
- A single request per epoch reveals one share—insufficient to recover sk.
- Two requests in the same epoch reveal two shares on a degree-1 polynomial, enabling recovery of sk and slashing the user's deposit.
- Different epochs produce different nullifiers, providing **per-request unlinkability**.

5.3 Circuit Composition

The key architectural insight: the VAnchor circuit (audited by Veridise [5], ∼38k constraints) and the RLN circuit (audited by yAcademy [4], ∼5k constraints) are both Groth16 over BN254 using Poseidon. Composing them preserves the audit guarantees of each sub-circuit. The new audit surface is only the glue logic (∼100 constraints):

Construction 1 (Composed VAnchor-RLN Circuit)**.** The composed circuit includes three sub-circuits:

Sub-circuit A (VAnchor, audited): Proves UTXO ownership, Merkle membership, nullifier correctness, and amount conservation. Outputs: cm, amount.

Glue logic (new, ∼100 constraints):

$$\mathsf{identityCommitment} := H(\mathsf{cm}) \tag{1}$$

$$(\mathsf{epochIndex} + 1) \times C_{\max} \leq \mathsf{amount} + R \quad \text{(solvency)} \tag{2}$$

where R is the accumulated refund balance and $C_{\max}$ is the maximum cost per request.

Sub-circuit B (RLN, audited): Takes $\mathsf{identityCommitment}$ as input. Produces $\mathsf{rlnNull} = H(\mathsf{sk}, \mathsf{epoch})$ and Shamir share. Enforces rate-limiting and enables slashing on double-signal.

The sub-circuit constraints are *unchanged*—the glue only connects outputs of A to inputs of B and adds the solvency check. Existing audit findings carry over to the composed circuit. Only the glue logic requires new review.

5.4 v2 Protocol Flow

1. **Deposit** (on-chain, one-time): User deposits stablecoins into the VAnchor pool.
2. **Per-request proof** (client-side, ∼640ms): User generates a composed VAnchor-RLN proof. The proof demonstrates: UTXO ownership, solvency, and produces an RLN nullifier + Shamir share.
3. **Operator verification** (off-chain, ∼5ms): Operator checks the proof against the on-chain Merkle root, verifies the nullifier is fresh, and stores the Shamir share.
4. **Service delivery**: Operator serves the request.
5. **Refund ticket**: Operator signs a refund ticket $r = (C_{\max} - C_{\mathsf{actual}}, \sigma_{\mathsf{server}})$ for unused capacity. The user accumulates refund tickets locally to unlock further requests without additional deposits.
6. **Batch settlement** (on-chain, periodic): Operator submits accumulated nullifiers and claims payment in bulk. No per-request on-chain transaction.
7. **Slashing**: If a user double-signals in an epoch, two Shamir shares are revealed. Anyone can recover sk and slash the user's deposit.

5.5 v1 → v2 Comparison

	v1 (Credits)	**v2 (RLN)**
Per-request proof	ECDSA signature	Groth16 (off-chain)
Per-request on-chain tx	∼50k gas	None (batched)
Per-request latency	<100ms	∼640ms (client)
Intra-session linkability	Yes (same C)	No (per-request nullifier)
Rate limiting	Nonce ordering	Shamir slashing
Refund for unused capacity	No	Yes (server-signed tickets)
New audit surface	1,000 LOC Solidity	∼100 circuit constraints

Both versions coexist—operators can accept either v1 SpendAuth or v2 RLN proofs. Users choose their privacy level per request.

6 Use Cases

The system targets workloads where *usage patterns* are as sensitive as the data itself.

LLM Inference. Developers querying LLMs for code review, legal analysis, or medical triage create prompt histories that reveal strategic intent. Shielded payments decouple the query from the querier.

Image and Video Generation. Creative professionals generating images or video may wish to keep their prompts and creative direction private from the service provider and from on-chain observers.

AI Agent Execution. Autonomous agents paying for inference, tool use, and compute should not reveal their controller's identity. The agent's on-chain address is decoupled from the entity that funds it.

Fine-Tuning. Organizations submitting proprietary datasets for model fine-tuning can prevent competitors from monitoring training activity via payment trails.

General Cloud Compute. Any Blueprint-based service—databases, CI/CD, storage, web hosting—can accept shielded payments. The credit system is service-agnostic.

7 Cross-Chain Architecture

The VAnchor supports cross-chain deposits via linked Merkle roots relayed through LayerZero V2. A user deposits USDC on Ethereum; the root is relayed to Base; the user withdraws on Base citing the Ethereum root in the membership proof.

The circuit compiles with $\mathsf{maxEdges} = 7$ (8 chains including self). Unoccupied edge slots are filled with deterministic zero hashes. Chains are added dynamically via `addEdge`—a single multisig transaction that sets the chain mapping, LayerZero peer, and initial Merkle root atomically.

Initial deployment: Ethereum, Arbitrum, Base, Hyperliquid, BSC. Room for 3 additional chains without circuit recompilation.

8 Cost Analysis

Approach	Per-request gas	Per-request latency	Privacy
ZK proof per call [2]	∼300k (on-chain)	∼640ms	Full
VAnchor withdrawal per call	∼300k (on-chain)	∼640ms	Full
v1: Shielded Credits	**∼50k (on-chain)**	**<100ms**	**Pseudonymous**
v2: RLN (off-chain)	**0 (batched)**	**∼640ms**	**Full**
x402 [7] (no privacy)	∼30k (on-chain)	<50ms	None

The v1 one-time proof cost is amortized over all jobs in a session. A \$50 credit account at \$0.10/call amortizes over 500 calls. The v2 proof cost is per-request but entirely client-side with zero on-chain gas.

9 Implementation

The v1 system is deployed as a standalone Foundry project at `tangle-network/shielded-payment-gateway`. It compiles against the audited protocol-solidity contracts (git submodule, unmodified) and requires no changes to Tangle's core protocol.

Component	Lines	Audit status
VAnchor + TokenWrapper + Merkle	∼3,000	Veridise [5]
RLN circuit (v2)	∼500	yAcademy [4]
OpenZeppelin 4.x / 5.x	—	OpenZeppelin
ShieldedGateway	188	New (v1)
ShieldedCredits	232	New (v1)
LayerZeroAnchorBridge	260	New (v1)
Interfaces	252	New (v1)
Glue constraints (v2)	∼100	Planned

Test coverage: 65 Forge tests (including 4 fuzz tests for financial invariants) and 40 TypeScript SDK tests (including real Groth16 proof generation against 8-edge production circuits in ∼640ms).

References

[1] D. Stone, "Webb Protocol: A cross-chain private application and governance protocol," *IACR ePrint 2023/260*, 2023.

[2] D. Crapis and V. Buterin, "ZK API Usage Credits: LLMs and Beyond," *ethresear.ch*, March 2026. https://ethresear.ch/t/zk-api-usage-credits-llms-and-beyond/24104

[3] B. Whitehat *et al.*, "Rate-Limiting Nullifiers," Privacy & Scaling Explorations, Ethereum Foundation. https://rate-limiting-nullifier.github.io/rln-docs/

[4] yAcademy, "Rate-Limiting Nullifier security review," 2023. https://hackmd.io/@yAcademy/rln-review

[5] Veridise Inc., "Security audit of Webb Protocol Solidity contracts," 2023.

[6] A. Pertsev, R. Semenov, and R. Storm, "Tornado Cash privacy solution," 2019.

[7] Coinbase, "x402: An open standard for internet-native payments," 2025. https://x402.org

[8] R. Floersch and B. Johnson, "EIP-712: Typed structured data hashing and signing," Ethereum Improvement Proposals, 2017.

[9] L. Grassi *et al.*, "Poseidon: A new hash function for zero-knowledge proof systems," *USENIX Security*, 2021.

[10] J. Groth, "On the size of pairing-based non-interactive arguments," *EUROCRYPT*, 2016.